

The Complete Google Colab Guide for Computer Vision Students

Your hands-on reference for mastering Google Colab in computer vision coursework. Updated for May 2026.

Course: ITAI 1378 Computer Vision for Artificial Intelligence

Program: AI and Robotics | Digital and Information Technology Center of Excellence

Institution: Houston City College

Author: Patricia McManus

Collaborator: Claude (Anthropic)

Version 2.0 | May 2026

Table of Contents

1. Introduction to Google Colab
2. Getting Started: Your First Steps
3. The Colab Interface
4. Working with Cells and Code
5. GPU Setup and Management
6. File Management and Storage
7. GitHub Integration
8. Hugging Face Integration
9. Dataset Management
10. Computer Vision Libraries
11. Gemini AI Code Assistant
12. Notebook Export and Sharing
13. Computer Vision Workflows
14. Advanced Topics
15. Troubleshooting Common Issues
16. Best Practices and Tips
17. Project Templates

NEW IN 2026

This is the 2026 revision of the Colab guide. Major changes from version 1.0 include updated GPU descriptions and pricing, fixed broken code examples, modern PyTorch and torchvision APIs, current Gemini and VLM model names, and the new YOLO toolkit. See the back of the guide for a full list.

QUICK CHECK

Accessibility statement: This document is designed for WCAG 2.1 AA compliance. It uses proper heading structure for screen reader navigation, color contrast ratios above the 4.5:1 minimum for body text, descriptive labels on all callouts (not just color), and 12-point body type. The included Python code examples follow the same accessibility principles you should apply to your own notebooks. See section 16 for student-facing accessibility guidance.

1. Introduction to Google Colab

What is Google Colab?

Imagine you need to complete a complex science experiment that requires an expensive, high-tech laboratory. Instead of buying everything, you are given free access to a state of the art lab over the internet. You log in, and all the equipment is there, ready to use.

Google Colab is that online lab, but for writing and running code.

It is a free service from Google that lets you write and run Python code directly in your browser. You do not need a powerful computer or any complicated setup. Most importantly for us, it gives you free access to specialized computer hardware called GPUs (Graphics Processing Units), which are necessary for modern Artificial Intelligence and Computer Vision tasks.

Each file you work on is called a notebook (.ipynb file). It is not just a plain code file. It is an interactive document where you can combine runnable code, formatted text, images, and links to explain your work.

Why Colab for Computer Vision?

KEY TAKEAWAY

Free GPU access: essential for training deep learning models

Pre-installed libraries: OpenCV, PyTorch, TensorFlow come ready to use

Cloud storage: integrates with Google Drive

Collaboration: share notebooks like Google Docs

No setup required: start coding immediately

Using Your Google Account with Colab

To use Colab, you need a Google account. Most students use a personal Gmail account.

Why use one consistent account

Think of your Google account as the key to your lab and your personal locker (your Google Drive). By always using the same Google account, all your Colab notebooks and project files are automatically saved and organized in one place. It is like having a dedicated backpack for school. You know exactly where your books and notes are every time.

TIP

If you want to keep your schoolwork separate from your personal files, consider creating a new, free Google account just for your studies. With your

2. Getting Started: Your First Steps

Step 1: Opening Your First Notebook

To create a new notebook

1. Go to colab.research.google.com
2. A pop-up window will appear. Click on the New notebook button.

To upload a notebook (for example, an assignment file)

3. Go to colab.research.google.com
4. In the pop-up window, click the Upload tab.
5. Drag and drop the .ipynb file or click Choose File to find it on your computer.

Step 2: Your First Code Cell

Start with a simple Hello World example.

```
# Your first computer vision code
print("Hello, Computer Vision World!")

# Check if we are in Colab
import sys
if "google.colab" in sys.modules:
    print("You are running in Google Colab.")
else:
    print("This does not appear to be Colab.")
```

TIP

Copy the code above into a new code cell and press Shift + Enter to run it.

Step 3: Connecting to a Runtime

Before you can run code, you must click the Connect button in the top-right corner. This assigns you a virtual computer with RAM and disk space.

WATCH OUT

Always connect before running any code cells. If a cell will not run, check the top right corner first.

3. The Colab Interface

When your notebook opens, you will see several key areas.

Menu Bar (Top)

- **File:** Create, open, save notebooks
- **Edit:** Cut, copy, paste cells
- **View:** Toggle sections, table of contents
- **Insert:** Add cells, code snippets
- **Runtime:** Manage your virtual computer (most important)
- **Tools:** Settings and keyboard shortcuts

Toolbar (Below Menu Bar)

Quick access buttons for adding code or text cells, running a cell, stopping execution, and restarting your runtime.

Left Sidebar Tabs

Table of Contents

Automatically generated from your text headings. Click to jump to sections.

Find and Replace

Search your entire notebook. Replace text across multiple cells.

Files

Most important tab. Shows all files in your session, lets you upload files, and lets you mount Google Drive.

Code Snippets

Ready-to-use code for common tasks. Great for beginners learning new functions.

Secrets

NEW IN 2026

The Secrets panel (the key icon) is where you store sensitive tokens like Hugging Face, GitHub, and Kaggle credentials. Always use this instead of hardcoding tokens in your notebook.

4. Working with Cells and Code

Understanding Cell Types

Your notebook is made of individual blocks called cells. You can have as many as you want.

Code cells

- Gray background with a play icon
- Write and run Python code here
- Results appear below the cell

Text cells (Markdown)

- White background
- Write notes, instructions, and explanations
- Use Markdown formatting

Essential Cell Operations

To create a cell

Hover your mouse in the space just below an existing cell. The + Code and + Text buttons will appear.

To run a cell

Click the play icon or select the cell and press Shift + Enter.

To delete a cell

Click on the cell and then click the trash can icon.

To move a cell

Click on the cell and use the up or down arrow buttons in the cell toolbar.

To change a cell type

Use the dropdown in the toolbar to switch between Code and Text.

Basic Markdown for Text Cells

Double-click a text cell to edit it. Use these simple codes to format your text.

```
# Heading 1 (a big title)
## Heading 2 (a smaller subtitle)
### Heading 3 (even smaller)
```

```
*italic text* or _italic text_  
**bold text** or __bold text__
```

```
* A bullet point  
- Another bullet point  
1. Numbered list item  
2. Another numbered item
```

```
[Link text](https://example.com)  
![Image alt text](image-url.jpg)
```

```
> A quote or note block
```

```
`inline code` for highlighting code words
```

Essential Keyboard Shortcuts

KEY TAKEAWAY

Shift + Enter: Run current cell and move to next

Ctrl + Enter: Run current cell and stay on it

Esc then M: Change current cell to Markdown

Esc then Y: Change current cell to code

Esc then A: Insert new cell Above

Esc then B: Insert new cell Below

Esc then D twice: Delete current cell

[↑ Back to top](#)

5. GPU Setup and Management

Why GPUs Matter for Computer Vision

CPUs (regular processors) are like a single very smart person working on problems one at a time. GPUs are like having hundreds of people working together on the same problem simultaneously. For computer vision tasks like training neural networks, processing large images, running deep learning models, and real-time video analysis, GPUs can be 10 to 100 times faster than CPUs.

Choosing Your Hardware Accelerator

Step 1: Access Runtime Settings

6. Go to the menu bar and click Runtime
7. Select Change runtime type

Step 2: Select Hardware Accelerator

You will see these options.

CPU: Default option, slowest for AI tasks. Use only for simple data processing.

GPU: Recommended for almost all computer vision work. Free tier typically gives you an NVIDIA T4 (16GB memory).

TPU: Google custom hardware. Very fast for specific tasks but less flexible. Most students stick with GPU.

Premium GPUs: A100 and L4 are available with paid subscriptions. You do not need these for coursework.

Step 3: Save Settings

Select GPU and click Save. Your runtime will restart. This is normal.

NEW IN 2026

Free GPU access in 2026 has weekly quotas (typically 15 to 30 GPU hours per week). Availability is not guaranteed during peak hours. The maximum session length is 12 hours and the idle timeout is 90 minutes.

Checking Your GPU

Run this code to verify your GPU setup.

```
# Check if GPU is available
import torch
import tensorflow as tf

print("Checking GPU availability...")
print(f"PyTorch CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU Device: {torch.cuda.get_device_name(0)}")
    print(f"GPU Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")

print(f"TensorFlow GPU devices: {len(tf.config.list_physical_devices(\"GPU\"))}")

# Check system info
!nvidia-smi
```

GPU Usage Limits and Best Practices

Free tier limitations (May 2026)

- 12-hour maximum runtime per session
- 90-minute idle timeout if no code is running
- Weekly GPU hour quotas (15 to 30 hours, varies by demand)
- During peak hours, free users may get CPU only or be disconnected

INDUSTRY PRACTICE

Switch your runtime to CPU when you are not actively training a model.

Save work frequently to Google Drive (your local session storage is wiped when you disconnect).

Use GPU only when necessary (training, inference, large image processing).

Monitor your usage with !nvidia-smi.

Upgrading to Colab Pro

If you need more GPU time:

Colab Pro: \$11.99 per month

- Priority GPU access
- Longer runtimes
- More memory

Colab Pro+: \$49.99 per month

- Premium GPUs (A100, L4)
- Background execution
- Highest priority access

Pay As You Go

- Buy compute units as needed instead of a monthly subscription
- Useful if you have one short, intensive project

TIP

For coursework, the free tier is enough for the majority of labs. Consider Pro only if you are running long fine-tuning jobs or training big models. Coursework labs are designed to fit within free tier limits.

[↑ Back to top](#)

6. File Management and Storage

Understanding Colab Storage

Think of Colab storage like a hotel room and a permanent home.

Temporary session storage (/content/)

- Your hotel room. Fast access, but everything is deleted when you check out (disconnect or session ends).
- Perfect for temporary files, downloads, and processing.

Google Drive (/content/drive/MyDrive/)

- Your permanent home. Persistent storage that survives session restarts.
- Where you save important work, datasets, and trained models.

Mounting Google Drive

Mounting connects your temporary session to your permanent Google Drive storage.

```
# Mount Google Drive
from google.colab import drive
drive.mount("/content/drive")

# Verify mounting worked
import os
print("Contents of your Google Drive:")
print(os.listdir("/content/drive/MyDrive")[:10]) # Show first 10 items
```

After running this code:

8. A popup window will open
9. Choose your Google account
10. Click Allow to grant Colab access to your Drive
11. The popup will close automatically

QUICK CHECK

You will see "Mounted at /content/drive" when it works.

NEW IN 2026

The Drive authorization flow no longer involves copying and pasting a long token. As of late 2021, Colab uses a standard OAuth popup. If you read older guides telling you to paste an authorization code, ignore that step.

File Operations

Uploading files from your computer

Method 1: Using the Files Panel

12. Click the Files icon on the left sidebar
13. Click the Upload button
14. Select files from your computer

Method 2: Using Code

```
# Upload files programmatically
from google.colab import files

# Upload one or multiple files
uploaded = files.upload()

# See what was uploaded
for filename in uploaded.keys():
    print(f"Uploaded: {filename} ({len(uploaded[filename])} bytes)")
```

Downloading files to your computer

```
from google.colab import files

# Download a specific file
files.download("my_results.csv")

# Or download from Google Drive
files.download("/content/drive/MyDrive/my_model.pth")
```

Working with directories

```
import os

# Create a project folder in your Drive
project_path = "/content/drive/MyDrive/CV_Project"
os.makedirs(project_path, exist_ok=True)

# Create subdirectories
os.makedirs(f"{project_path}/data", exist_ok=True)
```

```
os.makedirs(f"{project_path}/models", exist_ok=True)
os.makedirs(f"{project_path}/results", exist_ok=True)

print("Project structure created.")
print(os.listdir(project_path))
```

File path best practices

```
import os

# Good: absolute paths
data_path = "/content/drive/MyDrive/datasets/images"
model_path = "/content/drive/MyDrive/models/my_model.pth"

# Avoid: relative paths (they can break unexpectedly)
# data_path = "../datasets/images"

# Check if paths exist before using them
if os.path.exists(data_path):
    print(f"Found data at: {data_path}")
else:
    print(f"Data not found at: {data_path}")
```

Managing large files

```
import zipfile
import shutil

# Compress a folder before saving to Drive
def compress_folder(folder_path, zip_path):
    shutil.make_archive(zip_path.replace(".zip", ""), "zip", folder_path)
    print(f"Compressed {folder_path} to {zip_path}")

# Extract compressed files
def extract_zip(zip_path, extract_to):
    with zipfile.ZipFile(zip_path, "r") as zip_ref:
        zip_ref.extractall(extract_to)
    print(f"Extracted {zip_path} to {extract_to}")
```

[↑ Back to top](#)

7. GitHub Integration

Why Integrate GitHub with Colab

GitHub is a version control system and backup service for your code. Think of it as a library where you store all versions of your notebooks, a collaboration platform for team projects, and a way to sync work between different environments.

Setting Up GitHub Access

Method 1: Direct GitHub Integration (easiest)

Opening notebooks from GitHub

15. Go to colab.research.google.com
16. In the pop-up, click the GitHub tab
17. Enter a GitHub repository URL or search for repositories
18. Select the notebook you want to open

For private repositories

19. Check the Include Private Repos box
20. Sign in to GitHub when prompted
21. Grant Colab permission to access your repositories

Method 2: Using Git Commands (more control)

```
# Clone a public repository
!git clone https://github.com/username/repository-name.git

# For private repositories, use a Personal Access Token (PAT)
# Create one at: https://github.com/settings/tokens
!git clone https://YOUR_PAT@github.com/username/private-repo.git

# Navigate to the cloned repository
import os
os.chdir("repository-name")
print(f"Current directory: {os.getcwd()}")
```

Working with GitHub Repositories

Cloning your course repository

```
# Example: Clone your computer vision course repository
!git clone https://github.com/your-instructor/cv-course-2026.git

# Navigate to the repository
import os
os.chdir("cv-course-2026")

# List the contents
!ls -la
```

Saving changes back to GitHub

Method 1: Colab built-in GitHub save

22. After editing your notebook, go to File then Save a copy in GitHub
23. Select your repository and branch
24. Enter a commit message (for example, Completed Assignment 1)
25. Click OK

Method 2: Git commands

```
# Configure Git (do this once per session)
!git config --global user.email "your-email@example.com"
!git config --global user.name "Your Name"

# Add your changes
!git add .

# Commit with a message
!git commit -m "Added computer vision assignment solution"

# Push to GitHub (you will need authentication)
!git push origin main
```

GitHub Authentication with Personal Access Tokens

For secure authentication, use Personal Access Tokens stored in Colab Secrets.

```
# Store your PAT securely using Colab Secrets
from google.colab import userdata

# Get your stored PAT (add it in the Secrets panel first)
github_token = userdata.get("GITHUB_TOKEN")
```

```
# Clone with authentication
repo_url = f"https://{github_token}@github.com/username/repo-name.git"
!git clone {repo_url}
```

WATCH OUT

Never hardcode tokens in your notebooks. Always use the Colab Secrets feature (key icon in the left sidebar). If you accidentally commit a token to GitHub, revoke it immediately and create a new one.

Useful GitHub Plus Colab Tips

Open any GitHub notebook directly

You can open any public GitHub notebook by modifying the URL.

```
Original: https://github.com/user/repo/blob/main/notebook.ipynb
Colab:
https://colab.research.google.com/github/user/repo/blob/main/notebook.ipynb
```

Chrome extension for quick access

26. Search the Chrome Web Store for Open in Colab
27. Install the extension
28. You can now click the Colab icon on any GitHub notebook page

[↑ Back to top](#)

8. Hugging Face Integration

What is Hugging Face

Hugging Face is a massive library of pre-trained AI models. Think of it as a repository of ready-to-use AI models, a place to find datasets for training and testing, and a set of tools for building AI applications.

For computer vision, you will find models for:

- Image classification
- Object detection
- Image segmentation
- Image generation
- Vision-Language Models (VLMs)

Setting Up Hugging Face in Colab

Step 1: Install required libraries

```
# Install Hugging Face libraries
!pip install --upgrade huggingface_hub transformers datasets

# For computer vision tasks, also install:
!pip install --upgrade torch torchvision pillow
```

Step 2: Create and store your access token

29. Go to huggingface.co/settings/tokens
30. Create a new token with Read permissions
31. Copy the token

Store it securely in Colab

32. Click the Secrets icon (key) in the left sidebar
33. Add a new secret named HF_TOKEN
34. Paste your token as the value

Step 3: Authenticate in your notebook

```
# Using stored token (recommended)
from google.colab import userdata
from huggingface_hub import login

# Get your stored token
hf_token = userdata.get("HF_TOKEN")

# Login programmatically
login(hf_token)
print("Successfully logged in to Hugging Face.")
```

Using Hugging Face Models for Computer Vision

Example 1: Image classification

```
from transformers import pipeline
from PIL import Image
import requests

# Load a pre-trained image classification model
classifier = pipeline("image-classification",
                      model="google/vit-base-patch16-224")

# Load an image from URL
# Tip: bundle demo images in your course Drive folder if Unsplash links
break
url = "https://images.unsplash.com/photo-1574158622682-e40e69881006"
image = Image.open(requests.get(url, stream=True).raw)

# Classify the image
results = classifier(image)

print("Image Classification Results:")
for result in results:
    print(f"  {result[\"label\"]}: {result[\"score\"]:.3f}")
```

Example 2: Object detection

```
from transformers import pipeline
import matplotlib.pyplot as plt
import matplotlib.patches as patches

# Load object detection model
detector = pipeline("object-detection",
                    model="facebook/detr-resnet-50")
```

```

# Detect objects in the image
detections = detector(image)

# Visualize results
fig, ax = plt.subplots(1, 1, figsize=(12, 8))
ax.imshow(image)

for detection in detections:
    box = detection["box"]
    label = detection["label"]
    score = detection["score"]

    rect = patches.Rectangle(
        (box["xmin"], box["ymin"]),
        box["xmax"] - box["xmin"],
        box["ymax"] - box["ymin"],
        linewidth=2, edgecolor="red", facecolor="none"
    )
    ax.add_patch(rect)

    ax.text(box["xmin"], box["ymin"] - 10,
            f"{label}: {score:.2f}",
            color="red", fontsize=12, weight="bold")

ax.axis("off")
plt.title("Object Detection Results")
plt.show()

```

Downloading and using datasets

```

from datasets import load_dataset

# Load a computer vision dataset
dataset = load_dataset("cifar10")

print("Dataset Info:")
print(f"  Train samples: {len(dataset[\"train\"])}")
print(f"  Test samples: {len(dataset[\"test\"])}")
print(f"  Classes: {dataset[\"train\"].features[\"label\"].names}")

# Look at a sample
sample = dataset["train"][0]
print(f"Sample image shape: {sample[\"img\"].size}")

```

```
print(f"Label:
{dataset[\"train\"].features[\"label\"].names[sample[\"label\"]]})")
```

Uploading your own models

```
from huggingface_hub import HfApi, create_repo

# Create a new repository for your model
repo_name = "my-cv-model"
create_repo(repo_name, private=False)

# Upload your trained model
api = HfApi()

api.upload_file(
    path_or_fileobj="/content/drive/MyDrive/my_model.pth",
    path_in_repo="pytorch_model.bin",
    repo_id=f"your-username/{repo_name}",
)

print(f"Model uploaded to: https://huggingface.co/your-username/{repo_name}")
```

Common CV Pipelines

```
# Available computer vision pipelines
cv_tasks = [
    "image-classification",
    "object-detection",
    "image-segmentation",
    "depth-estimation",
    "image-to-text",
    "text-to-image",
    "zero-shot-image-classification",
]

print("Available Computer Vision Tasks:")
for task in cv_tasks:
    print(f"  - {task}")
```

[↑ Back to top](#)

9. Dataset Management

Overview of Dataset Sources

For computer vision projects, you will work with datasets from various sources.

- Local files: images from your computer
- Kaggle: competitive datasets and challenges
- Hugging Face: curated ML datasets
- Google Drive: your personal storage
- Public URLs: direct image downloads
- Built-in datasets: from libraries like torchvision

Method 1: Uploading Local Files

Small files (under 25 MB)

```
from google.colab import files
import os

print("Select images to upload:")
uploaded = files.upload()

# Process uploaded files
for filename in uploaded.keys():
    print(f"Uploaded: {filename}")

    # Move to an organized folder
    os.makedirs("/content/uploaded_images", exist_ok=True)
    os.rename(filename, f"/content/uploaded_images/{filename}")

print(f"Files saved to: /content/uploaded_images")
print(f"Total files: {len(os.listdir('/content/uploaded_images'))}")
```

Large files via Google Drive

```
from google.colab import drive
import os, shutil

# Mount Drive
drive.mount("/content/drive")
```

```
# Copy dataset from Drive to local storage for faster access
source_path = "/content/drive/MyDrive/datasets/my_images"
local_path = "/content/dataset"

if os.path.exists(source_path):
    shutil.copytree(source_path, local_path)
    print(f"Dataset copied to local storage: {local_path}")
    print(f"Total files: {len(os.listdir(local_path))}")
else:
    print(f"Dataset not found at: {source_path}")
```

Method 2: Kaggle Datasets

NEW IN 2026

The modern way to use Kaggle from Colab is via the Secrets panel, not by uploading kaggle.json each time. This matches how you authenticate with Hugging Face and GitHub.

Recommended setup using Colab Secrets

First, get your Kaggle credentials.

35. Go to kaggle.com/settings and click Create New Token
36. Download kaggle.json (contains your username and key)
37. Open the file and copy the username and key values

Then, store both in Colab Secrets.

38. Click the Secrets icon (key) in the left sidebar
39. Add a secret named KAGGLE_USERNAME with your username as the value
40. Add a secret named KAGGLE_KEY with your key as the value

Now you can use Kaggle without managing files.

```
from google.colab import userdata
import os

# Set Kaggle credentials from Secrets
os.environ["KAGGLE_USERNAME"] = userdata.get("KAGGLE_USERNAME")
os.environ["KAGGLE_KEY"] = userdata.get("KAGGLE_KEY")

# Install Kaggle API
!pip install -q kaggle

print("Kaggle API configured.")
```

Download Kaggle datasets

```
# Search for datasets
!kaggle datasets list -s "computer vision"

# Download a specific dataset
dataset_name = "puneet6060/intel-image-classification"
!kaggle datasets download -d {dataset_name}

# Extract the dataset
import zipfile
with zipfile.ZipFile(f"{dataset_name.split('/')[-1]}.zip", "r") as zip_ref:
    zip_ref.extractall("/content/kaggle_dataset")

print("Kaggle dataset downloaded and extracted.")
print(f"Contents: {os.listdir('/content/kaggle_dataset')}")
```

Method 3: Built-in Datasets

```
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# Download CIFAR-10 dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(
    root="/content/data",
    train=True,
    download=True,
    transform=transform,
)

testset = torchvision.datasets.CIFAR10(
    root="/content/data",
    train=False,
    download=True,
    transform=transform,
)

print(f"CIFAR-10 Dataset:")
```

```

print(f" Training samples: {len(trainset)}")
print(f" Test samples: {len(testset)}")
print(f" Classes: {trainset.classes}")

# Create data loaders
trainloader = DataLoader(trainset, batch_size=32, shuffle=True)
testloader = DataLoader(testset, batch_size=32, shuffle=False)

```

Data Augmentation Setup

```

import torchvision.transforms as transforms

# Define augmentation pipeline
train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

val_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

print("Data augmentation pipeline created.")

```

[↑ Back to top](#)

10. Computer Vision Libraries

Essential Libraries

Computer vision in Python relies on several key libraries, each serving different purposes.

- OpenCV: image processing and computer vision algorithms
- PyTorch: deep learning framework with torchvision
- TensorFlow / Keras: Google deep learning framework
- scikit-image: scientific image processing
- Matplotlib: plotting and visualization
- Pillow (PIL): basic image operations
- NumPy: numerical computing foundation

Installation and Setup

```
# Install essential computer vision libraries
!pip install --upgrade opencv-python opencv-contrib-python
!pip install --upgrade torch torchvision torchaudio
!pip install --upgrade tensorflow
!pip install --upgrade scikit-image
!pip install --upgrade matplotlib pillow numpy

print("All essential CV libraries installed.")
```

Verify Installations

```
import cv2, torch, torchvision, skimage, matplotlib, PIL
import tensorflow as tf
import numpy as np

print("Library Versions:")
print(f"  OpenCV: {cv2.__version__}")
print(f"  PyTorch: {torch.__version__}")
print(f"  TorchVision: {torchvision.__version__}")
print(f"  TensorFlow: {tf.__version__}")
print(f"  scikit-image: {skimage.__version__}")
print(f"  Matplotlib: {matplotlib.__version__}")
print(f"  Pillow: {PIL.__version__}")
print(f"  NumPy: {np.__version__}")
```

```
# Check GPU availability
print(f"\nHardware:")
print(f"  PyTorch CUDA: {torch.cuda.is_available()}")
print(f"  TensorFlow GPU: {len(tf.config.list_physical_devices(\"GPU\")) > 0}")
```

OpenCV Basics

Basic image operations

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Download a sample image
!wget -O sample.jpg https://images.unsplash.com/photo-1574158622682-e40e69881006

# Read image
img = cv2.imread("sample.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Convert BGR to RGB

print(f"Image shape: {img.shape}")
print(f"  Height: {img.shape[0]} pixels")
print(f"  Width: {img.shape[1]} pixels")
print(f"  Channels: {img.shape[2]}")

# Display image
plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.subplot(1, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Image filtering and enhancement

```

# Apply various filters
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

axes[0, 0].imshow(img_rgb)
axes[0, 0].set_title("Original")
axes[0, 0].axis("off")

blurred = cv2.GaussianBlur(img_rgb, (15, 15), 0)
axes[0, 1].imshow(blurred)
axes[0, 1].set_title("Gaussian Blur")
axes[0, 1].axis("off")

edges = cv2.Canny(gray, 100, 200)
axes[0, 2].imshow(edges, cmap="gray")
axes[0, 2].set_title("Canny Edges")
axes[0, 2].axis("off")

equalized = cv2.equalizeHist(gray)
axes[1, 0].imshow(equalized, cmap="gray")
axes[1, 0].set_title("Histogram Equalized")
axes[1, 0].axis("off")

kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(edges, cv2.MORPH_OPEN, kernel)
axes[1, 1].imshow(opening, cmap="gray")
axes[1, 1].set_title("Morphological Opening")
axes[1, 1].axis("off")

contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
contour_img = img_rgb.copy()
cv2.drawContours(contour_img, contours, -1, (0, 255, 0), 2)
axes[1, 2].imshow(contour_img)
axes[1, 2].set_title(f"Contours ({len(contours)} found)")
axes[1, 2].axis("off")

plt.tight_layout()
plt.show()

```

PyTorch for Computer Vision

Basic tensor operations

```
import torch
```

```

import torchvision.transforms as transforms
from PIL import Image

# Load and preprocess image
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

pil_image = Image.open("sample.jpg")
tensor_image = transform(pil_image)

print(f"Tensor shape: {tensor_image.shape}")
print(f" Channels: {tensor_image.shape[0]}")
print(f" Height:   {tensor_image.shape[1]}")
print(f" Width:    {tensor_image.shape[2]}")
print(f" Data type: {tensor_image.dtype}")

# Move to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tensor_image = tensor_image.to(device)
print(f"Tensor device: {tensor_image.device}")

```

Pre-trained models (modern API)

NEW IN 2026

torchvision deprecated the pretrained=True parameter in 2022 and now uses the weights parameter. The new API lets you choose which set of pretrained weights you want and avoids deprecation warnings.

```

from torchvision.models import resnet50, ResNet50_Weights

# Modern API: use weights parameter
weights = ResNet50_Weights.IMAGENET1K_V2
model = resnet50(weights=weights) # or weights="DEFAULT"
model.eval()
model = model.to(device)

print(f"Model: {model.__class__.__name__}")
print(f" Parameters: {sum(p.numel() for p in model.parameters()),}")
print(f" Device: {next(model.parameters()).device}")

```

```
# Make a prediction
with torch.no_grad():
    input_tensor = tensor_image.unsqueeze(0) # Add batch dimension
    output = model(input_tensor)

    probabilities = torch.nn.functional.softmax(output[0], dim=0)
    top5_prob, top5_idx = torch.topk(probabilities, 5)

# Use the weights' built-in category labels (no need to download a file)
categories = weights.meta["categories"]

print(f"\nTop 5 Predictions:")
for i in range(5):
    class_name = categories[top5_idx[i]]
    probability = top5_prob[i].item()
    print(f"  {i+1}. {class_name}: {probability:.3f}")
```

TensorFlow / Keras Setup

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print(f"TensorFlow version: {tf.__version__}")
print(f"GPU available: {tf.config.list_physical_devices(\"GPU\")}")

def create_cnn_model(input_shape=(224, 224, 3), num_classes=10):
    model = keras.Sequential([
        layers.Conv2D(32, (3, 3), activation="relu",
input_shape=input_shape),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation="relu"),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation="relu"),
        layers.Flatten(),
        layers.Dense(64, activation="relu"),
        layers.Dense(num_classes, activation="softmax")
    ])
    return model

model = create_cnn_model()
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

```
)  
  
model.summary()
```

Scikit-Image for Advanced Processing

WATCH OUT

In older versions of this guide, the watershed example called functions that do not exist. The corrected code below uses `scipy.ndimage.distance_transform_edt` and `skimage.feature.peak_local_max`. Always check the import lines first.

```
from scipy.ndimage import distance_transform_edt  
from skimage import filters, segmentation, measure  
from skimage.feature import peak_local_max  
from skimage.color import rgb2gray  
import numpy as np  
import matplotlib.pyplot as plt  
from PIL import Image  
  
# Convert image for scikit-image  
pil_image = Image.open("sample.jpg")  
sk_image = np.array(pil_image)  
sk_gray = rgb2gray(sk_image)  
  
# Otsu thresholding  
thresh = filters.threshold_otsu(sk_gray)  
binary = sk_image > thresh  
  
# Watershed segmentation (corrected imports)  
distance = distance_transform_edt(binary)  
coords = peak_local_max(distance, min_distance=20)  
  
mask = np.zeros(distance.shape, dtype=bool)  
mask[tuple(coords.T)] = True  
markers, _ = measure.label(mask, return_num=True)  
labels = segmentation.watershed(-distance, markers, mask=binary)  
  
# Visualize  
fig, axes = plt.subplots(1, 3, figsize=(15, 5))  
axes[0].imshow(sk_image)  
axes[0].set_title("Original")  
axes[0].axis("off")  
  
axes[1].imshow(binary, cmap="gray")
```

```

axes[1].set_title("Otsu Threshold")
axes[1].axis("off")

axes[2].imshow(labels, cmap="nipy_spectral")
axes[2].set_title("Watershed Segmentation")
axes[2].axis("off")

plt.tight_layout()
plt.show()

```

Performance Optimization

```

import torch
import tensorflow as tf

def optimize_for_gpu():
    """Configure optimal GPU settings"""
    # PyTorch optimizations
    if torch.cuda.is_available():
        torch.backends.cudnn.benchmark = True
        print("PyTorch GPU optimizations enabled.")

    # TensorFlow optimizations
    gpus = tf.config.experimental.list_physical_devices("GPU")
    if gpus:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
            print("TensorFlow GPU memory growth enabled.")

    return torch.cuda.is_available(), len(gpus) > 0

pytorch_gpu, tf_gpu = optimize_for_gpu()

```

11. Gemini AI Code Assistant

What is Gemini in Colab

Gemini is Google AI assistant integrated directly into Colab to help you write, debug, and understand code. Think of it as having an expert programmer ready to:

- Generate code from natural language descriptions
- Explain complex code in simple terms
- Debug errors and suggest fixes
- Analyze data and create visualizations
- Optimize performance and suggest improvements

NEW IN 2026

As of May 2026, Colab uses the Gemini 3 family. Current options include Gemini 3.1 Pro Preview (highest quality) and Gemini 3.5 Flash (faster, lower cost). Model names change quickly. Check the Gemini menu in your Colab notebook to see what is currently available to you.

Accessing Gemini in Colab

Method 1: Gemini Spark Icon

Look for the spark icon in your Colab interface.

41. Click the spark icon in the toolbar
42. Type your request in natural language
43. Gemini will generate code or explanations

Method 2: Data Science Agent

The Data Science Agent is a powerful feature for automated analysis.

44. Click the Gemini icon in the left sidebar
45. Upload your dataset
46. Describe what you want to analyze
47. Gemini will generate complete analysis code

TIP

Some advanced Gemini features in Colab depend on your account region and your subscription tier. If a feature does not appear, check your account settings.

Practical Examples with Gemini

Example 1: Generating computer vision code

Prompt: "Create a function to load and preprocess images for training."

Gemini might generate something like this:

```
import torch
from torchvision import transforms
from PIL import Image
import os

def load_and_preprocess_images(image_dir, image_size=(224, 224)):
    """Load and preprocess images for training."""
    transform = transforms.Compose([
        transforms.Resize(image_size),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                             std=[0.229, 0.224, 0.225])
    ])

    images = []
    valid_extensions = (".jpg", ".jpeg", ".png", ".bmp", ".tiff")

    for filename in os.listdir(image_dir):
        if filename.lower().endswith(valid_extensions):
            try:
                image_path = os.path.join(image_dir, filename)
                image = Image.open(image_path).convert("RGB")
                tensor_image = transform(image)
                images.append(tensor_image)
            except Exception as e:
                print(f"Error processing {filename}: {e}")

    print(f"Loaded {len(images)} images from {image_dir}")
    return images
```

Example 2: Debugging with Gemini

If you get an error like `RuntimeError: Expected 4-dimensional input for 4-dimensional weight`, ask Gemini why and it will explain plus provide a fix.

```
def fix_tensor_dimensions(tensor):
    """Fix common tensor dimension issues for CNN input."""
    print(f"Original tensor shape: {tensor.shape}")

    # If tensor is 3D (C, H, W), add batch dimension
    if len(tensor.shape) == 3:
        tensor = tensor.unsqueeze(0)
        print(f"Added batch dimension: {tensor.shape}")

    # If tensor is 2D (H, W), add channel and batch dimensions
    elif len(tensor.shape) == 2:
        tensor = tensor.unsqueeze(0).unsqueeze(0)
        print(f"Added channel and batch dimensions: {tensor.shape}")

    # If tensor has wrong channel order (B, H, W, C), permute to (B, C, H, W)
    elif len(tensor.shape) == 4 and tensor.shape[3] == 3:
        tensor = tensor.permute(0, 3, 1, 2)
        print(f"Reordered to NCHW: {tensor.shape}")

    return tensor
```

Best Practices for Using Gemini

Effective prompting

Good prompts for Gemini

- Create a data augmentation pipeline for medical images
- Explain why my CNN is overfitting and suggest solutions
- Generate code to visualize feature maps from a trained model
- How do I implement transfer learning with ResNet50
- Debug this error: CUDA out of memory during training

Less effective prompts

- Fix my code (too vague)
- Make it better (no specific goal)
- Computer vision (too broad)

TIP

Be specific about your goal. Include relevant context (dataset, model, error message). Ask for explanations, not just code. Request best practices and optimizations.

Validating Gemini suggestions

Always validate Gemini code before relying on it.

- Does the code run without errors
- Are the results reasonable and expected
- Is the code following best practices
- Are there any security or performance concerns
- Does it handle edge cases properly

WATCH OUT

Gemini is powerful but not perfect. Always review and test generated code thoroughly. Watch for outdated APIs in particular: Gemini may suggest `pretrained=True` (deprecated) or `torch.cuda.amp` (legacy). Cross-check the imports.

12. Notebook Export and Sharing

Why Export Notebooks

Exporting your Colab notebooks is essential for:

- Submitting assignments in PDF format
- Sharing work with classmates and instructors
- Creating portfolios for job applications
- Archiving projects for future reference
- Version control and backup

Method 1: Browser Print to PDF (recommended)

This is the quickest, most reliable method, requiring no additional setup.

Pre-export checklist

- All cells have been run and show outputs
- Text cells are properly formatted with Markdown
- Images and plots are displaying correctly
- Code is commented and readable

Steps for Browser Export

48. Clean up your notebook: run all cells to ensure outputs are current
49. Press Ctrl+P (or Cmd+P on Mac) to open the print dialog
50. Select Save as PDF (or Microsoft Print to PDF on Windows)
51. Adjust settings if needed (margins, scale)
52. Save the file with a descriptive name

TIP

This is the most reliable method for class submissions. It works in every browser and never breaks when Google updates Colab.

Method 2: Using nbconvert (high quality)

For professional-quality PDFs with proper formatting:

```
# Install required packages
!pip install nbconvert
```

```
!apt-get install -y texlive-xetex texlive-fonts-recommended pandoc

print("nbconvert and dependencies installed.")
```

```
import os
from google.colab import drive

# Mount Drive to save the PDF
drive.mount("/content/drive")

# Replace with your actual notebook name
notebook_name = "your_notebook_name.ipynb"

# Convert to PDF
!jupyter nbconvert --to pdf "{notebook_name}"

# Move PDF to Google Drive for permanent storage
pdf_name = notebook_name.replace(".ipynb", ".pdf")
!cp "{pdf_name}" "/content/drive/MyDrive/"

print(f"PDF saved to Google Drive: {pdf_name}")
```

Method 3: HTML Export

Sometimes HTML export works better than PDF, especially for notebooks with rich visualizations.

```
!jupyter nbconvert --to html "your_notebook.ipynb"

print("HTML file created.")
print("Open the HTML file in a browser and use Ctrl+P to save as PDF if needed.")
```

Sharing Your Notebooks

Sharing via Google Drive

```
from google.colab import drive
import shutil
import os

drive.mount("/content/drive")

# Create a shared folder for your class
```

```

shared_folder = "/content/drive/MyDrive/CV_Class_Shared"
os.makedirs(shared_folder, exist_ok=True)

# Copy your notebook to the shared folder
notebook_name = "Assignment_1_YourName.ipynb"
shutil.copy(notebook_name, f"{shared_folder}/{notebook_name}")

print("Notebook copied to shared folder.")
print(f"Location: {shared_folder}")
print("Right-click the folder in Google Drive to share with others.")

```

Sharing via GitHub

```

!git config --global user.email "your-email@example.com"
!git config --global user.name "Your Name"

# Add and commit your notebook
!git add "Assignment_1.ipynb"
!git commit -m "Completed Assignment 1: Image Classification"
!git push origin main

print("Notebook pushed to GitHub.")

```

Creating Professional Reports

```

from IPython.display import HTML, display
from datetime import datetime

def create_report_header(title, author, course):
    """Create a professional header for your notebook."""
    today = datetime.now().strftime("%B %d, %Y")
    header_html = f"""
    <div style="border: 2px solid #4CAF50; padding: 20px; margin: 20px 0;
background-color: #f9f9f9;">
        <h1 style="color: #2E7D32; margin: 0;">{title}</h1>
        <hr style="border: 1px solid #4CAF50;">
        <p><strong>Author:</strong> {author}</p>
        <p><strong>Course:</strong> {course}</p>
        <p><strong>Date:</strong> {today}</p>
        <p><strong>Environment:</strong> Google Colab</p>
    </div>
    """
    display(HTML(header_html))

# Example usage

```

```
create_report_header(  
    title="Computer Vision Assignment 1: Image Classification",  
    author="Your Name",  
    course="ITAI 1378 Computer Vision",  
)
```

TIP

The example above generates today's date automatically. Older guides used a hardcoded date string that quickly became outdated. Using `datetime.now()` means the example header always shows the current date when run.

Export Quality Tips

- Use high-resolution images (300 DPI for print)
- Set figure sizes explicitly: `plt.figure(figsize=(10, 6))`
- Use clear, readable fonts and colors
- Add descriptive titles and captions to all plots
- Format numbers consistently: `f"{value:.3f}"`
- Use markdown headers to structure your content
- Add explanatory text between code cells
- Remove or hide debugging print statements
- Keep code lines under 80 to 100 characters
- Run all cells before exporting to ensure fresh outputs

13. Computer Vision Workflows

This section demonstrates end-to-end computer vision workflows you will use throughout your coursework.

Workflow 1: Image Classification from Scratch

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision.transforms as transforms
from PIL import Image
import os
import matplotlib.pyplot as plt

# Step 1: Custom Dataset Class
class CustomImageDataset(Dataset):
    """Custom dataset for loading images from class-named folders."""
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.classes = sorted(os.listdir(root_dir))
        self.class_to_idx = {cls: idx for idx, cls in
            enumerate(self.classes)}

        self.images = []
        self.labels = []

        for class_name in self.classes:
            class_dir = os.path.join(root_dir, class_name)
            if os.path.isdir(class_dir):
                for img_name in os.listdir(class_dir):
                    if img_name.lower().endswith((".png", ".jpg",
".jpeg")):
                        self.images.append(os.path.join(class_dir,
img_name))
                        self.labels.append(self.class_to_idx[class_name])

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_path = self.images[idx]
```

```

        image = Image.open(img_path).convert("RGB")
        label = self.labels[idx]
        if self.transform:
            image = self.transform(image)
        return image, label

```

Define data transforms with augmentation.

```

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

```

Define a simple CNN.

```

class SimpleCNN(nn.Module):
    """Simple CNN for image classification."""
    def __init__(self, num_classes):
        super().__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),

```

```

        nn.MaxPool2d(2, 2),

        nn.Conv2d(128, 256, kernel_size=3, padding=1),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2, 2),
    )

    self.classifier = nn.Sequential(
        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Flatten(),
        nn.Dropout(0.5),
        nn.Linear(256, 128),
        nn.ReLU(inplace=True),
        nn.Dropout(0.3),
        nn.Linear(128, num_classes)
    )

    def forward(self, x):
        x = self.conv_layers(x)
        x = self.classifier(x)
        return x

```

Training loop.

```

def train_model(model, train_loader, val_loader, num_epochs=10):
    """Train the model with validation."""
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=7,
gamma=0.1)

    history = {"train_loss": [], "train_acc": [], "val_loss": [],
"val_acc": []}

    for epoch in range(num_epochs):
        print(f"\nEpoch {epoch+1}/{num_epochs}")

        # Training phase
        model.train()
        train_loss, train_correct, train_total = 0.0, 0, 0

```

```

for data, targets in train_loader:
    data, targets = data.to(device), targets.to(device)
    optimizer.zero_grad()
    outputs = model(data)
    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()
    train_loss += loss.item()
    _, predicted = outputs.max(1)
    train_total += targets.size(0)
    train_correct += predicted.eq(targets).sum().item()

# Validation phase
model.eval()
val_loss, val_correct, val_total = 0.0, 0, 0

with torch.no_grad():
    for data, targets in val_loader:
        data, targets = data.to(device), targets.to(device)
        outputs = model(data)
        loss = criterion(outputs, targets)
        val_loss += loss.item()
        _, predicted = outputs.max(1)
        val_total += targets.size(0)
        val_correct += predicted.eq(targets).sum().item()

scheduler.step()

# Log metrics
train_loss /= len(train_loader)
val_loss /= len(val_loader)
train_acc = 100.0 * train_correct / train_total
val_acc = 100.0 * val_correct / val_total
history["train_loss"].append(train_loss)
history["train_acc"].append(train_acc)
history["val_loss"].append(val_loss)
history["val_acc"].append(val_acc)
print(f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
print(f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")

return model, history

```

Workflow 2: Transfer Learning Pipeline

INDUSTRY PRACTICE

Transfer learning is the standard approach in production. You almost never train from scratch. You start with a pretrained model and fine-tune the last layers (or a few extra layers) for your specific dataset.

```
from torchvision.models import resnet50, ResNet50_Weights
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights
import torch.nn as nn

def create_transfer_learning_model(num_classes, model_name="resnet50"):
    """Create a transfer learning model (modern weights API)."""

    if model_name == "resnet50":
        weights = ResNet50_Weights.IMAGENET1K_V2
        model = resnet50(weights=weights)

        # Freeze early layers
        for param in model.parameters():
            param.requires_grad = False

        # Replace final layer
        num_features = model.fc.in_features
        model.fc = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(num_features, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes)
        )

    elif model_name == "efficientnet":
        weights = EfficientNet_B0_Weights.DEFAULT
        model = efficientnet_b0(weights=weights)

        for param in model.parameters():
            param.requires_grad = False

        num_features = model.classifier[1].in_features
        model.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(num_features, num_classes)
        )
```

```
return model
```

Workflow 3: Object Detection Pipeline

```
from transformers import DetrImageProcessor, DetrForObjectDetection
import torch
from PIL import Image, ImageDraw
import matplotlib.pyplot as plt

def setup_object_detection():
    """Set up DETR object detection model."""
    processor = DetrImageProcessor.from_pretrained("facebook/detr-resnet-50")
    model = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")
    return processor, model

def detect_objects(image_path, processor, model, threshold=0.9):
    """Detect objects in an image."""
    image = Image.open(image_path)
    inputs = processor(images=image, return_tensors="pt")
    outputs = model(**inputs)

    target_sizes = torch.tensor([image.size[::-1]])
    results = processor.post_process_object_detection(
        outputs, target_sizes=target_sizes, threshold=threshold
    )[0]

    draw = ImageDraw.Draw(image)
    detections = []
    for score, label, box in zip(results["scores"], results["labels"],
results["boxes"]):
        box = [round(i, 2) for i in box.tolist()]
        label_name = model.config.id2label[label.item()]
        draw.rectangle(box, outline="red", width=3)
        draw.text((box[0], box[1]), f"{label_name}: {round(score.item(),
3)}", fill="red")
        detections.append({
            "label": label_name,
            "confidence": round(score.item(), 3),
            "box": box,
        })
```

```
return image, detections
```

Evaluation and Metrics

```
from sklearn.metrics import accuracy_score,
precision_recall_fscore_support, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import torch

def evaluate_model(model, test_loader, class_names):
    """Comprehensive model evaluation."""
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.eval()

    all_predictions = []
    all_targets = []

    with torch.no_grad():
        for data, targets in test_loader:
            data, targets = data.to(device), targets.to(device)
            outputs = model(data)
            _, predicted = outputs.max(1)
            all_predictions.extend(predicted.cpu().numpy())
            all_targets.extend(targets.cpu().numpy())

    accuracy = accuracy_score(all_targets, all_predictions)
    precision, recall, f1, support = precision_recall_fscore_support(
        all_targets, all_predictions, average=None
    )
    cm = confusion_matrix(all_targets, all_predictions)

    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=class_names, yticklabels=class_names)
    plt.title("Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

    print(f"Overall Accuracy: {accuracy:.4f}")
    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
```

```

    "f1": f1,
    "confusion_matrix": cm,
}

```

Model Save and Load (modern PyTorch)

NEW IN 2026

PyTorch 2.6+ changed the default of `torch.load` to `weights_only=True` for security. When you load checkpoints saved with the older full-state format (model + optimizer + history together), you must explicitly pass `weights_only=False`. The corrected examples below handle both modes.

```

import torch

def save_model_for_deployment(model, model_path, class_names):
    """Save model with metadata for deployment."""
    checkpoint = {
        "model_state_dict": model.state_dict(),
        "class_names": class_names,
        "num_classes": len(class_names),
    }
    torch.save(checkpoint, model_path)
    print(f"Model saved to: {model_path}")

def load_model_for_inference(model_path, model_class):
    """Load model for inference."""
    # weights_only=False because we are loading a dict with non-tensor
    metadata
    # (class_names, num_classes). Only do this for checkpoints you trust.
    checkpoint = torch.load(
        model_path, map_location="cpu", weights_only=False
    )

    num_classes = checkpoint["num_classes"]
    model = model_class(num_classes)
    model.load_state_dict(checkpoint["model_state_dict"])
    model.eval()

    class_names = checkpoint["class_names"],
    print(f"Model loaded from: {model_path}")
    return model, class_names

```

[↑ Back to top](#)

14. Advanced Topics

Vision-Language Models (VLMs)

Vision-Language Models combine computer vision and natural language processing to understand both images and text. By 2026 these are the most workforce-relevant CV models. They are also the foundation for the AI agent project later in this course.

NEW IN 2026

The 2026 landscape of VLMs: closed frontier models include GPT-5.5 / GPT-5.5 Pro, Claude Opus 4.7, and Gemini 3. Open-weight options include Mistral Large 3, Qwen3-VL, InternVL3.5, Molmo 2, and LLaVA-style systems. Model names move quickly; always check the latest before any lab.

Image captioning and VQA with Florence-2

Florence-2 from Microsoft is the open-weight VLM most commonly used in workforce settings in 2026. It is small, fast, and handles captioning, visual question answering, and detection in one model.

```
from transformers import (
    AutoProcessor,
    AutoModelForCausalLM,
)
from PIL import Image
import torch

def setup_florence():
    """Set up Florence-2 for captioning and VQA."""
    model_id = "microsoft/Florence-2-large"
    processor = AutoProcessor.from_pretrained(model_id,
trust_remote_code=True)
    model = AutoModelForCausalLM.from_pretrained(
        model_id, trust_remote_code=True,
        torch_dtype=torch.float16 if torch.cuda.is_available() else
torch.float32,
    )
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)
    return processor, model, device

def generate_caption(image_path, processor, model, device):
    """Generate caption for an image."""
```

```

image = Image.open(image_path).convert("RGB")
prompt = "<MORE_DETAILED_CAPTION>"
inputs = processor(text=prompt, images=image,
return_tensors="pt").to(device)

with torch.no_grad():
    generated_ids = model.generate(
        input_ids=inputs["input_ids"],
        pixel_values=inputs["pixel_values"],
        max_new_tokens=200,
        num_beams=3,
    )

    generated_text = processor.batch_decode(generated_ids,
skip_special_tokens=False)[0]
    parsed = processor.post_process_generation(
        generated_text, task=prompt, image_size=(image.width, image.height)
    )
    return parsed[prompt]

```

DEEPER LOOK

Older guides used Salesforce/blip-image-captioning-base for this kind of work. BLIP from 2022 still loads and runs but is clearly behind modern models. Florence-2, Qwen3-VL, and similar produce significantly better captions. For coursework BLIP is fine if you want to see the simple API; for any real project, reach for Florence-2 or a frontier closed VLM.

VLM via closed API (production reality)

In most real workforce settings, you do not load a VLM into your notebook. You call a hosted API. The pattern looks like this.

```

# Example: calling a frontier VLM API (pseudocode pattern)
# Replace with the actual SDK for your chosen provider:
#   - openai (for GPT-5.5 family)
#   - anthropic (for Claude Opus 4.7)
#   - google-generativeai (for Gemini 3 family)
#   - mistralai (for Mistral Large 3)

# Most providers follow a similar pattern: encode the image,
# send it with a text prompt, parse the text response.

import base64
from pathlib import Path

def encode_image_base64(image_path):

```

```

    return base64.b64encode(Path(image_path).read_bytes()).decode()

image_b64 = encode_image_base64("sample.jpg")
# Then pass image_b64 plus a prompt to your provider's chat/messages
endpoint.
# Each provider has slightly different field names; see the provider SDK
docs.

```

3D Computer Vision

Understanding 3D space is crucial for advanced computer vision applications such as robotics, AR/VR, and depth-aware photography.

Monocular depth estimation

Depth Anything V2 is the current standard open-weight model for depth estimation in 2026.

```

from transformers import pipeline
from PIL import Image
import matplotlib.pyplot as plt

# Modern depth estimator
depth_estimator = pipeline(
    "depth-estimation",
    model="depth-anything/Depth-Anything-V2-Large-hf",
)

# Use any image: image = Image.open("your_image.jpg")
image = Image.open("sample.jpg")
depth = depth_estimator(image)

# Visualize
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
ax1.imshow(image)
ax1.set_title("Original")
ax1.axis("off")

ax2.imshow(depth["depth"], cmap="plasma")
ax2.set_title("Depth Map")
ax2.axis("off")

plt.tight_layout()
plt.show()

```

Generative Models

WATCH OUT

Older versions of this guide used `runwayml/stable-diffusion-v1-5`. RunwayML removed that checkpoint from Hugging Face in 2024. Use a community mirror, SDXL, or FLUX instead.

Image generation with FLUX (recommended)

FLUX from Black Forest Labs is the current go-to open-weight image generator. It produces high-quality output and the `schnell` variant is fast enough for class demonstrations.

```
from diffusers import FluxPipeline
import torch

def setup_flux():
    """Set up FLUX for image generation."""
    model_id = "black-forest-labs/FLUX.1-schnell"
    pipe = FluxPipeline.from_pretrained(
        model_id,
        torch_dtype=torch.bfloat16 if torch.cuda.is_available() else
torch.float32,
    )
    if torch.cuda.is_available():
        pipe.enable_model_cpu_offload() # Saves VRAM
    return pipe

def generate_images(pipe, prompts, num_images=1):
    """Generate images from text prompts."""
    results = []
    for prompt in prompts:
        print(f"Generating: {prompt}")
        images = pipe(
            prompt,
            num_inference_steps=4,      # FLUX schnell needs only a few steps
            guidance_scale=0.0,        # schnell ignores guidance
            num_images_per_prompt=num_images,
        ).images
        results.extend(images)
        for img in images:
            img.show()
    return results
```

Alternative: Stable Diffusion XL

If FLUX is too heavy for your free-tier GPU, SDXL is a solid choice.

```
from diffusers import StableDiffusionXLPipeline
import torch

model_id = "stabilityai/stable-diffusion-xl-base-1.0"
pipe = StableDiffusionXLPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float16 if torch.cuda.is_available() else
torch.float32,
)
if torch.cuda.is_available():
    pipe = pipe.to("cuda")

prompt = "a futuristic city with flying cars, cinematic lighting"
image = pipe(prompt, num_inference_steps=25).images[0]
image.show()
```

AI Agents for Computer Vision

Building intelligent agents that can perceive and act in visual environments is a major direction in 2026. The pattern combines a VLM (the perception layer) with planning, memory, and tool use.

NEW IN 2026

Examples of 2026 vision agents include Anthropic Computer Use, OpenAI ChatGPT agent and Operator, Microsoft OmniParser, and ByteDance UI-TARS. All use a VLM as the eyes, paired with action loops. Module 09 of this course covers these in depth.

```
from dataclasses import dataclass
from typing import Dict, List, Any
from datetime import datetime

@dataclass
class AgentAction:
    """Represents an action an agent can take."""
    action_type: str
    parameters: Dict[str, Any]
    confidence: float

class ComputerVisionAgent:
    """A simple perceive-reason-act agent for CV tasks."""
```

```

def __init__(self, name):
    self.name = name
    self.capabilities = []
    self.memory = []
    self.tools = {}

def add_capability(self, capability_name, capability_function):
    """Add a new capability to the agent."""
    self.capabilities.append(capability_name)
    self.tools[capability_name] = capability_function
    print(f"Agent {self.name} learned: {capability_name}")

def perceive(self, image_path):
    """Perceive and analyze an image."""
    perception = {
        "image_path": image_path,
        "timestamp": datetime.now().isoformat(),
        "analysis": {},
    }
    for capability in self.capabilities:
        if capability in self.tools:
            try:
                perception["analysis"][capability] =
self.tools[capability](image_path)
            except Exception as e:
                perception["analysis"][capability] = f"Error: {e}"
    self.memory.append(perception)
    return perception

def reason(self, perception):
    """Decide what to do based on perception."""
    actions = []
    analysis = perception["analysis"]
    if "object_detection" in analysis:
        objects = analysis["object_detection"]
        if len(objects) > 5:
            actions.append(AgentAction(
                action_type="alert",
                parameters={"message": f"Busy scene with {len(objects)}
objects"},
                confidence=0.8,
            ))
    return actions

```

```
def act(self, actions):
    """Execute the planned actions."""
    for action in actions:
        print(f"Agent {self.name} executing: {action.action_type}")
        print(f"  Parameters: {action.parameters}")
        print(f"  Confidence: {action.confidence}")

def run(self, image_path):
    """Complete perceive-reason-act loop."""
    perception = self.perceive(image_path)
    actions = self.reason(perception)
    self.act(actions)
    return perception, actions
```

[↑ Back to top](#)

15. Troubleshooting Common Issues

GPU and Runtime Issues

Problem: CUDA out of memory

Solutions in order of effort.

- Reduce batch size (most effective)
- Enable mixed precision training
- Use gradient accumulation
- Clear GPU cache between operations
- Reduce model size or image resolution
- Use CPU for some preprocessing steps

```
import torch, gc

def clear_gpu_memory():
    """Clear GPU memory cache."""
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
        gc.collect()
        print("GPU memory cleared.")

def monitor_gpu_memory():
    """Monitor current GPU memory usage."""
    if torch.cuda.is_available():
        allocated = torch.cuda.memory_allocated() / 1024**3
        cached = torch.cuda.memory_reserved() / 1024**3
        total = torch.cuda.get_device_properties(0).total_memory / 1024**3
        print(f"GPU Memory: {allocated:.2f}/{total:.2f} GB allocated,
{cached:.2f} GB cached")

def adaptive_batch_size(model, max_batch=64):
    """Find the largest batch size that fits in memory."""
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)
    batch_size = max_batch
    while batch_size > 1:
        try:
            dummy = torch.randn(batch_size, 3, 224, 224, device=device)
            with torch.no_grad():
                _ = model(dummy)
```

```

        print(f"Optimal batch size: {batch_size}")
        return batch_size
    except RuntimeError as e:
        if "out of memory" in str(e):
            batch_size //= 2
            torch.cuda.empty_cache()
            print(f"Trying batch size: {batch_size}")
        else:
            raise
    return batch_size

```

Problem: Mixed precision training (modern import)

NEW IN 2026

The torch.cuda.amp namespace still works but is legacy. The modern device-agnostic path is torch.amp (works for both CPU and GPU). Use this in new code.

```

from torch.amp import autocast, GradScaler
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
scaler = GradScaler("cuda")

for data, targets in train_loader:
    data, targets = data.to(device), targets.to(device)
    optimizer.zero_grad(set_to_none=True)

    with autocast(device_type="cuda"):
        outputs = model(data)
        loss = criterion(outputs, targets)

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()

```

Problem: Runtime disconnection

- Save checkpoints frequently to Google Drive
- Use the new auto-checkpoint feature in current Colab versions
- Implement resume training functionality
- Keep your browser tab active during long training

```
import torch, os
```

```
def save_checkpoint(epoch, model, optimizer, loss, save_dir):
    """Save a checkpoint."""
    os.makedirs(save_dir, exist_ok=True)
    checkpoint = {
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "loss": loss,
    }
    path = f"{save_dir}/checkpoint_epoch_{epoch}.pth"
    torch.save(checkpoint, path)
    print(f"Checkpoint saved: {path}")
    return path

def resume_training(model, optimizer, checkpoint_path):
    """Resume training from checkpoint."""
    if not os.path.exists(checkpoint_path):
        print("No checkpoint found, starting from scratch.")
        return 0, float("inf")

    # Use weights_only=False because we are loading the full training state
    checkpoint = torch.load(checkpoint_path, weights_only=False)
    model.load_state_dict(checkpoint["model_state_dict"])
    optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
    start_epoch = checkpoint["epoch"] + 1
    loss = checkpoint["loss"]
    print(f"Resumed from epoch {start_epoch}")
    return start_epoch, loss
```

Data Loading Issues

Problem: File not found errors

```
import os

def debug_file_paths():
    """Debug common file path issues."""
    print(f"Current working directory: {os.getcwd()}")
    print("Contents of current directory:")
    for item in sorted(os.listdir("."):
        marker = "[dir]" if os.path.isdir(item) else "[file]"
        print(f"  {marker} {item}")

    if os.path.exists("/content/drive"):
        print("Google Drive is mounted.")
```

```

        if os.path.exists("/content/drive/MyDrive"):
            items = len(os.listdir("/content/drive/MyDrive"))
            print(f"MyDrive has {items} top-level items.")
        else:
            print("Google Drive not mounted.")

def find_files(filename, search_path="."):
    """Find files matching a name recursively."""
    matches = []
    for root, dirs, files in os.walk(search_path):
        for file in files:
            if filename.lower() in file.lower():
                matches.append(os.path.join(root, file))
    print(f"Found {len(matches)} matches for {filename}.")
    for m in matches[:10]:
        print(f"  {m}")
    return matches

```

Problem: Image loading errors

```

from PIL import Image
import os

def validate_images(image_dir):
    """Validate all images in a directory."""
    valid, corrupted = [], []
    supported = (".png", ".jpg", ".jpeg", ".bmp", ".tiff", ".webp")

    for root, dirs, files in os.walk(image_dir):
        for file in files:
            path = os.path.join(root, file)
            if not file.lower().endswith(supported):
                continue
            try:
                with Image.open(path) as img:
                    img.verify()
                with Image.open(path) as img:
                    valid.append({"path": path, "size": img.size, "mode":
img.mode})
            except Exception as e:
                corrupted.append({"path": path, "error": str(e)})

    print(f"Valid images: {len(valid)}, Corrupted: {len(corrupted)}")
    return valid, corrupted

```

Model Training Issues

Problem: Loss not decreasing

Common causes and fixes:

- Learning rate too high: try 0.001 or 0.0001 with Adam
- Learning rate too low: try 0.01 or 0.001
- Vanishing gradients: use batch normalization, check activation functions
- Exploding gradients: use gradient clipping
- Data not normalized: check input range, apply normalization
- Labels wrong: print a few samples to verify
- Model too small: try a deeper or pretrained model

Library and Import Issues

```
def check_library_versions():
    """Check versions of all important libraries."""
    libraries = [
        "torch", "torchvision", "tensorflow", "cv2", "PIL", "numpy",
        "matplotlib", "sklearn", "transformers", "datasets",
    ]
    print("Library Version Check:")
    for lib in libraries:
        try:
            if lib == "cv2":
                import cv2
                v = cv2.__version__
            elif lib == "PIL":
                import PIL
                v = PIL.__version__
            elif lib == "sklearn":
                import sklearn
                v = sklearn.__version__
            else:
                module = __import__(lib)
                v = module.__version__
            print(f" OK  {lib}: {v}")
        except ImportError:
            print(f" --  {lib}: not installed")

def fix_import_issues():
    """Common fixes for import issues."""
```

```
print("Common import fixes (paste each in a new cell):")
print(" !pip install --upgrade torch torchvision")
print(" !pip install --upgrade tensorflow")
print(" !pip install opencv-python opencv-contrib-python")
print(" !pip install --upgrade transformers datasets")
print(" !pip install --upgrade scikit-learn scikit-image")
print(" !apt-get update && apt-get install -y libgl1")
```

WATCH OUT

When you should restart your runtime: after installing new packages, when getting import errors after an install, after CUDA out of memory errors, and when models behave unexpectedly. To restart: Runtime then Restart runtime.

[↑ Back to top](#)

16. Best Practices and Tips

Code Organization and Documentation

Project structure

```
import os

def create_project_structure():
    """Create a well-organized project structure."""
    structure = {
        "data": ["raw", "processed", "external"],
        "notebooks": ["exploratory", "experiments", "final"],
        "src": ["models", "data", "visualization", "utils"],
        "models": ["trained", "checkpoints"],
        "results": ["figures", "reports", "metrics"],
        "docs": [],
        "tests": [],
    }
    base = "/content/drive/MyDrive/CV_Project"
    for folder, subfolders in structure.items():
        os.makedirs(os.path.join(base, folder), exist_ok=True)
        for sub in subfolders:
            os.makedirs(os.path.join(base, folder, sub), exist_ok=True)
    print("Project structure created.")
```

Documentation best practices

- Use descriptive function names (preprocess_image, not process)
- Include docstrings with Args, Returns, Raises
- Add inline comments for complex logic
- Provide usage examples in docstrings
- Document expected input and output shapes for tensors

Code style

- Follow PEP 8 style guide
- Keep lines under 80 to 100 characters
- Use descriptive variable names (img_tensor, not x)
- Use type hints for function parameters
- Import libraries at the top of cells

- Remove unused imports and variables

Accessibility in Your Notebooks

When you submit work or share notebooks, make them accessible. Accessibility is required under ADA (Americans with Disabilities Act) and Section 508 for federally-funded institutions like ours, and it is simply good practice.

Code and output accessibility

- Add docstrings and comments. Screen readers will read them.
- Print labeled output. Instead of just `print(value)`, use `print(f"Accuracy: {value:.3f}")`.
- Do not use color alone to convey meaning. If your plot uses red for class A and green for class B, also use different shapes or labels.
- Provide text descriptions of complex visualizations in a Markdown cell next to the plot.

Image and plot accessibility

Every plot you generate should have:

- A descriptive title set with `plt.title("...")`
- Labeled axes set with `plt.xlabel("...")` and `plt.ylabel("...")`
- A legend if there are multiple data series
- Colors with sufficient contrast (avoid red-on-green or yellow-on-white)
- A text summary nearby in a Markdown cell describing what the plot shows

```
import matplotlib.pyplot as plt

# Accessible plotting example
plt.figure(figsize=(10, 6))
plt.plot(epochs, train_acc, label="Training", color="#1A237E", linewidth=2,
marker="o")
plt.plot(epochs, val_acc, label="Validation", color="#C62828", linewidth=2,
marker="s", linestyle="--")

# Two visual cues per series: color AND marker shape
# Also use different line styles (solid vs dashed)

plt.title("Model Accuracy Over Training Epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (percent)")
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Then in a Markdown cell explain what the plot shows in words:
# "Training accuracy reached 92 percent by epoch 20 while validation
# accuracy plateaued at 87 percent, indicating modest overfitting."
```

Document accessibility for assignments

- Use Markdown headings (# for H1, ## for H2) so screen readers can navigate the notebook
- Use descriptive section titles, not generic ones like "Step 1"
- When exporting to PDF for submission, set the document title in the properties
- Add alt text descriptions to any embedded images in Markdown cells: ![Description of image](url)

KEY TAKEAWAY

Accessibility is not extra work added at the end. It is good documentation, good labeling, good visualization. The accessibility version of your notebook is almost always also the clearer, more professional version. Build the habit early.

Performance Optimization

Optimized data loading

```
from torch.utils.data import DataLoader

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=2,          # Multiple workers for parallel loading
    pin_memory=True,        # Faster GPU transfer
    persistent_workers=True, # Keep workers alive between epochs
    prefetch_factor=2,      # Prefetch batches
)
```

- Use num_workers=2 to 4 for faster loading
- Set pin_memory=True for GPU training
- Use persistent_workers=True to avoid worker recreation
- Cache preprocessed data when possible

- Resize images to model input size early in the pipeline

Training optimization

- Use mixed precision training (torch.amp.autocast plus GradScaler)
- Accumulate gradients for larger effective batch size when memory is tight
- Use learning rate scheduling
- Enable torch.backends.cudnn.benchmark = True for fixed input sizes
- Use non_blocking=True for GPU transfers
- Freeze early layers in transfer learning
- Monitor GPU utilization with nvidia-smi

Reproducibility

```
import torch, numpy as np, random, os

def set_seed(seed=42):
    """Set seed for reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    os.environ["PYTHONHASHSEED"] = str(seed)
    print(f"Random seed set to {seed}")
```

- Set random seeds for all libraries
- Record library versions for every experiment
- Save model architecture and hyperparameters with the checkpoint
- Document data preprocessing steps
- Use version control (Git)
- Save training logs and metrics

Learning Resources

Foundational books

- Computer Vision: Algorithms and Applications by Richard Szeliski

- Deep Learning by Goodfellow, Bengio, and Courville
- Hands-On Computer Vision with TensorFlow 2 by Benjamin Planche

Online courses

- CS231n: Convolutional Neural Networks for Visual Recognition (Stanford)
- Fast.ai Practical Deep Learning for Coders
- Hugging Face Computer Vision course (free, online)

Foundational papers

- ImageNet Classification with Deep CNNs (AlexNet, 2012)
- Deep Residual Learning for Image Recognition (ResNet, 2015)
- Attention Is All You Need (Transformer, 2017)
- An Image is Worth 16x16 Words (Vision Transformer, 2020)
- Learning Transferable Visual Models from Natural Language Supervision (CLIP, 2021)

Communities

- r/MachineLearning on Reddit
- Papers With Code
- Hugging Face Forums
- Computer Vision Foundation (CVF) and CVPR proceedings

Debugging Mindset

- Read error messages carefully (the last line is usually the key)
- Check data shapes and types first
- Isolate the problem to a minimal example
- Add print statements to inspect intermediate values
- Test with simple, known inputs
- Search the exact error message online
- Ask for help with specific questions, not just "it does not work"
- Document solutions in your own notes for next time

[↑ Back to top](#)

17. Project Templates

Use these templates as a starting point for your coursework projects. Each one applies the modern APIs covered in earlier sections.

Template 1: Image Classification Project

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision.transforms as transforms
from torchvision.models import resnet50, ResNet50_Weights
from PIL import Image
import os
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
from tqdm import tqdm

class ImageClassificationProject:
    """Complete template for image classification projects."""

    def __init__(self, project_name, data_dir, num_classes):
        self.project_name = project_name
        self.data_dir = data_dir
        self.num_classes = num_classes
        self.device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
        self.history = {"train_loss": [], "train_acc": [], "val_loss": [],
"val_acc": []}
        print(f"Project {project_name} initialized on {self.device}")

    def setup_transforms(self):
        self.train_transform = transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(degrees=15),
            transforms.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                std=[0.229, 0.224, 0.225])
        ])
```

```

        self.val_transform = transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                std=[0.229, 0.224, 0.225])
        ])

    def create_model(self):
        """Create a transfer-learning ResNet50 with the modern weights
API."""
        weights = ResNet50_Weights.IMAGENET1K_V2
        self.model = resnet50(weights=weights)
        num_features = self.model.fc.in_features
        self.model.fc = nn.Linear(num_features, self.num_classes)
        self.model = self.model.to(self.device)
        self.optimizer = optim.Adam(self.model.parameters(), lr=0.001,
weight_decay=1e-4)
        self.criterion = nn.CrossEntropyLoss()
        print(f"Model created. Parameters: {sum(p.numel() for p in
self.model.parameters()):,}")

    def save_checkpoint(self, filename):
        checkpoint = {
            "model_state_dict": self.model.state_dict(),
            "optimizer_state_dict": self.optimizer.state_dict(),
            "num_classes": self.num_classes,
            "history": self.history,
        }
        torch.save(checkpoint, filename)
        print(f"Saved: {filename}")

    def load_checkpoint(self, filename):
        # weights_only=False for the full training state
        checkpoint = torch.load(filename, map_location=self.device,
weights_only=False)
        self.model.load_state_dict(checkpoint["model_state_dict"])
        self.optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
        self.history = checkpoint["history"]
        print(f"Loaded: {filename}")

```

Template 2: Object Detection Project (modern YOLO)

NEW IN 2026

Earlier guides used `pip install yolov5` (a third-party wrapper). The official Ultralytics package supports YOLOv8 and YOLO11 and is the workforce standard in 2026. This template uses ultralytics.

```
# Install once per session
# !pip install -q ultralytics

from ultralytics import YOLO
import torch

class ObjectDetectionProject:
    """Template for object detection projects using modern YOLO."""

    def __init__(self, project_name):
        self.project_name = project_name
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.model = None
        print(f"Object Detection Project: {project_name} on {self.device}")

    def load_pretrained(self, weights="yolo11n.pt"):
        """Load a pretrained YOLO model (nano, small, medium, large, extra-large)."""
        # Options: yolo11n.pt, yolo11s.pt, yolo11m.pt, yolo11l.pt, yolo11x.pt
        self.model = YOLO(weights)
        print(f"Loaded {weights}")

    def detect(self, image_path, conf=0.25):
        """Run detection on a single image."""
        results = self.model(image_path, conf=conf)
        results[0].show() # Display the annotated image
        return results[0]

    def batch_detect(self, image_dir):
        """Run detection on every image in a folder."""
        import os
        supported = (".png", ".jpg", ".jpeg")
        outputs = []
        for fn in sorted(os.listdir(image_dir)):
            if fn.lower().endswith(supported):
                path = os.path.join(image_dir, fn)
                result = self.detect(path)
```

```

        outputs.append((fn, result))
    return outputs

def train_custom(self, data_yaml, epochs=50, imgsz=640):
    """Train a custom YOLO model on your own dataset."""
    # data_yaml is a YAML file describing your dataset
    # See: https://docs.ultralytics.com/datasets/detect/
    self.model.train(
        data=data_yaml,
        epochs=epochs,
        imgsz=imgsz,
        device=self.device,
    )
    print("Training complete.")

```

Template 3: Computer Vision Agent Project

Use this as the starting point for your final agent project.

```

from transformers import (
    DetrImageProcessor, DetrForObjectDetection,
    AutoProcessor, AutoModelForCausalLM,
)
from PIL import Image
import torch
from datetime import datetime

class CVAgentProject:
    """Computer vision agent with perception, reasoning, and action."""

    def __init__(self, agent_name):
        self.agent_name = agent_name
        self.capabilities = []
        self.tools = {}
        self.memory = []
        self.device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
        print(f"Agent {agent_name} created on {self.device}")

    def add_object_detection(self):
        processor = DetrImageProcessor.from_pretrained(
            "facebook/detr-resnet-50"
        )
        model = DetrForObjectDetection.from_pretrained(
            "facebook/detr-resnet-50"

```

```

        ).to(self.device)

    def detect(image_path):
        image = Image.open(image_path).convert("RGB")
        inputs = processor(images=image,
return_tensors="pt").to(self.device)
        outputs = model(**inputs)
        target_sizes = torch.tensor([image.size[::-1]])
        results = processor.post_process_object_detection(
            outputs, target_sizes=target_sizes, threshold=0.9
        )[0]
        return [
            {
                "label": model.config.id2label[label.item()],
                "confidence": score.item(),
                "box": box.tolist(),
            }
            for score, label, box in zip(results["scores"],
results["labels"], results["boxes"])
        ]

    self.tools["object_detection"] = detect
    self.capabilities.append("object_detection")
    print("Added: object_detection")

    def add_captioning_florence(self):
        """Use Florence-2 for image captioning (modern VLM)."""
        model_id = "microsoft/Florence-2-large"
        processor = AutoProcessor.from_pretrained(model_id,
trust_remote_code=True)
        model = AutoModelForCausalLM.from_pretrained(
            model_id, trust_remote_code=True,
            torch_dtype=torch.float16 if torch.cuda.is_available() else
torch.float32,
        ).to(self.device)

    def caption(image_path):
        image = Image.open(image_path).convert("RGB")
        prompt = "<MORE_DETAILED_CAPTION>"
        inputs = processor(text=prompt, images=image,
return_tensors="pt").to(self.device)
        with torch.no_grad():
            generated_ids = model.generate(
                input_ids=inputs["input_ids"],
                pixel_values=inputs["pixel_values"],

```

```

        max_new_tokens=200,
        num_beams=3,
    )
    generated_text = processor.batch_decode(generated_ids,
skip_special_tokens=False)[0]
    parsed = processor.post_process_generation(
        generated_text, task=prompt, image_size=(image.width,
image.height)
    )
    return parsed[prompt]

self.tools["captioning"] = caption
self.capabilities.append("captioning")
print("Added: captioning (Florence-2)")

def perceive(self, image_path):
    perception = {
        "image_path": image_path,
        "timestamp": datetime.now().isoformat(),
        "analysis": {},
    }
    for cap in self.capabilities:
        try:
            perception["analysis"][cap] = self.tools[cap](image_path)
        except Exception as e:
            perception["analysis"][cap] = f"Error: {e}"
    self.memory.append(perception)
    return perception

def reason(self, perception):
    actions = []
    analysis = perception["analysis"]
    if "object_detection" in analysis:
        n_objects = len(analysis["object_detection"])
        if n_objects > 5:
            actions.append({
                "type": "alert",
                "message": f"Busy scene: {n_objects} objects detected",
                "confidence": 0.8,
            })
    return actions

def act(self, actions):
    for action in actions:
        print(f"[ACTION] {action[\"type\"]}: {action[\"message\"]}")

```

```
def run(self, image_path):  
    perception = self.perceive(image_path)  
    actions = self.reason(perception)  
    self.act(actions)  
    return perception, actions
```

Conclusion

You now have a comprehensive, current reference for Google Colab in computer vision coursework. This 2026 revision applies modern APIs, fixes known broken code, and reflects the current AI landscape.

What You Have Learned

- Colab fundamentals: interface, cells, GPU setup
- File management: Google Drive integration, dataset handling
- GitHub integration: version control and collaboration
- Hugging Face: pre-trained models and datasets
- Computer vision libraries: OpenCV, PyTorch (modern weights API), TensorFlow, scikit-image (corrected imports)
- Advanced topics: VLMs (Florence-2, Qwen3-VL, GPT-5.5, Claude Opus 4.7, Gemini 3), 3D CV, generative models (FLUX, SDXL), AI agents
- Best practices: code organization, debugging, optimization
- Project templates: image classification, modern YOLO detection, vision agents

Next Steps

53. Practice: start with the beginner exercises and work up
54. Experiment: try the project templates with your own data
55. Collaborate: use GitHub to share your work with classmates
56. Build: create your final agent project using the advanced techniques
57. Learn: continue exploring new computer vision developments

Final Tips

- Save frequently to Google Drive
- Use version control with GitHub
- Ask for help when stuck
- Document your work for future reference
- Keep experimenting and learning

KEY TAKEAWAY

The best way to learn computer vision is by doing. Start with simple projects, make mistakes, debug them, and gradually tackle more complex challenges. This guide is your roadmap, but your journey is unique.

What Changed in Version 2.0 (May 2026)

Major corrections and updates from version 1.0 (August 2025):

- Institution name corrected to Houston City College
- Broken skimage code on page 30 fixed (scipy.ndimage and skimage.feature)
- Replaced the dead runwayml/stable-diffusion-v1-5 with FLUX and SDXL
- All pretrained=True calls replaced with the modern weights API
- YOLOv5 wrapper replaced with the official Ultralytics package (YOLO11)
- Google Drive auth flow updated to the current OAuth popup
- Colab Pro pricing updated to \$11.99 per month
- Free GPU description includes weekly hour quotas
- torch.load() calls updated for PyTorch 2.6+ (weights_only handling)
- torch.cuda.amp imports updated to torch.amp
- BLIP captioning replaced with Florence-2 as the default modern VLM
- Depth estimation updated to Depth Anything V2
- Gemini section updated for the Gemini 3 family
- Kaggle setup now uses Colab Secrets for consistency
- Hardcoded date in the report header example replaced with datetime.now()
- Removed the Python 3.7 path reference
- Quarto-based PDF script demoted; browser print-to-PDF is the recommended method
- Made WCAG 2.1 AA compliant: 12pt body text, contrast ratios verified, descriptive callout labels (no color-only meaning), proper heading structure, document language set
- Added a new Accessibility in Your Notebooks subsection to section 16 covering accessible plots, output, and Markdown structure for student work

This guide was created for the AI and Robotics Program, Digital and Information Technology Center of Excellence, at Houston City College. For questions or updates, please refer to your course materials or contact your instructor.

Author: Patricia McManus

Collaborator: Claude (Anthropic)

Version: 2.0

Updated: May 2026

Environment: Google Colab